

Aura v2 Technical Documentation

1. Introduction

Aura v2 is a Python-based desktop assistant that expands on the architecture of Aura v1.

The primary improvements in this generation are:

- Persistent memory
- Encrypted memory storage
- Automatic encryption key generation
- Password-based Redx Mode authentication
- Protected Redx key storage
- A dedicated Redx Command Center
- Multi-file project architecture

Aura v2 is built around a Tkinter graphical interface and local command processing.

2. Technology Stack

Core Language

Python

GUI

Tkinter

Encryption Library

cryptography.fernet

Standard Library Components

os
json

```
base64
hashlib
```

3. Project Files

aura_gen2.py

The main Aura v2 application.

Responsibilities include:

- GUI creation
 - Command processing
 - Memory encryption
 - Memory persistence
 - Redx Mode authentication
 - Redx Command Center
-

keygen.py

A utility for generating the protected Redx key.

The script:

1. Generates a Fernet key
 2. Derives an encryption key from a password
 3. Encrypts the generated Fernet key
 4. Saves the encrypted result
-

redx_key.key

The encryption key used for Aura's persistent memory system.

Aura automatically loads this file when available.

If it does not exist, Aura generates a new Fernet key.

redx_key_protected.key

Contains the encrypted Redx key.

The key is unlocked when the supplied password successfully decrypts the protected file.

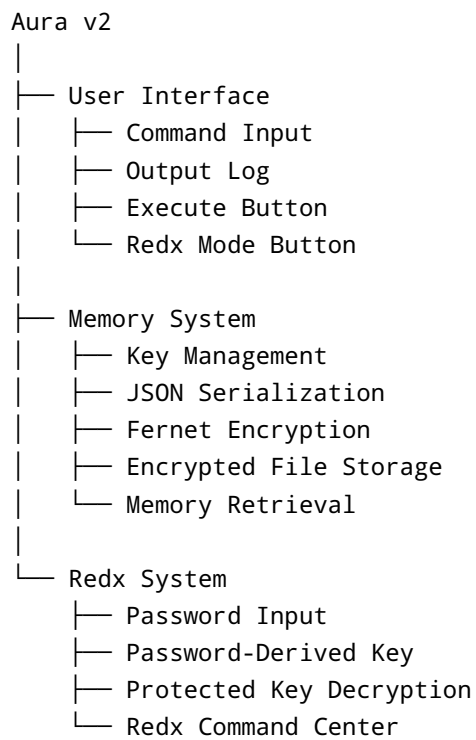
redx_memory.enc

Contains Aura's encrypted persistent memory.

The memory file stores encrypted JSON data.

4. System Architecture

Aura v2 contains three major subsystems.



5. Password-Derived Key System

Aura v2 includes a function responsible for converting a password into a valid Fernet-compatible key.

The process is conceptually:

```
graph TD; Password --> UTF8[UTF-8 Encoding]; UTF8 --> SHA256[SHA-256 Hash]; SHA256 --> Base64[Base64 URL-Safe Encoding]; Base64 --> Fernet[Fernet-Compatible Key];
```

The resulting key is used to decrypt the protected Redx key file.

6. Protected Redx Key System

Redx Mode uses a separate protected key file.

Authentication flow:

```
graph TD; A[User Requests Redx Mode] --> B[Password Dialog Opens]; B --> C[User Enters Password]; C --> D[Password-Derived Key Created]; D --> E[Attempt to Decrypt Protected Key]; E --> F[Success]; E --> G[Failure]; F --> H[Activate Mode]; G --> I[Access Denied]; H --> J[Open Command Center];
```

7. keygen.py

The key generation utility creates the protected Redx key.

Its process is:

```
User Password
    ↓
Generate Random Fernet Key
    ↓
Derive Key From Password
    ↓
Encrypt Fernet Key
    ↓
Save Protected Key File
```

The resulting file is:

```
redx_key_protected.key
```

The key-generation utility uses Fernet encryption and SHA-256-derived password processing.

8. Encrypted Memory System

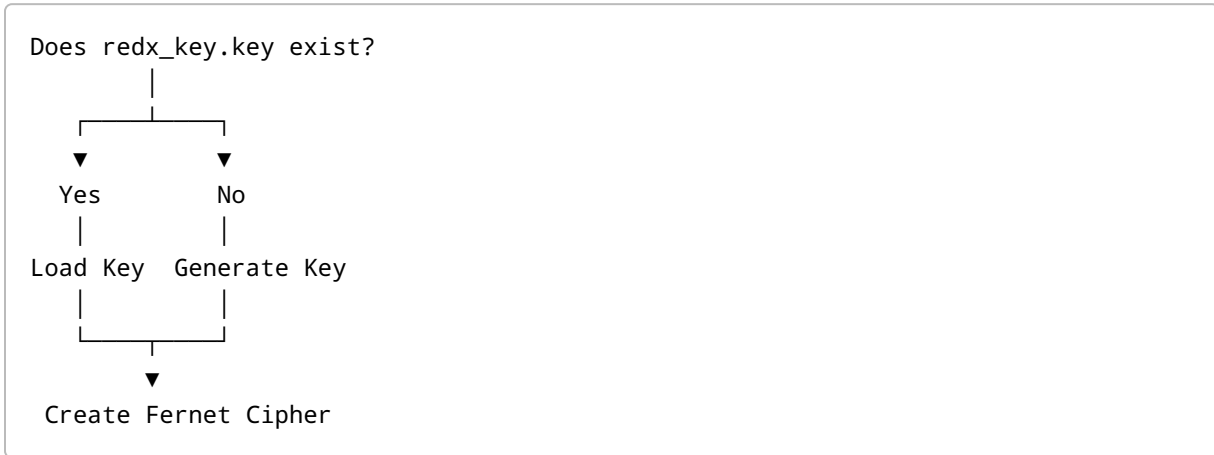
Aura v2 stores memory using encrypted local storage.

The memory system has three major stages:

```
Memory Dictionary
    ↓
JSON Serialization
    ↓
Fernet Encryption
    ↓
Encrypted File
```

9. Memory Key Management

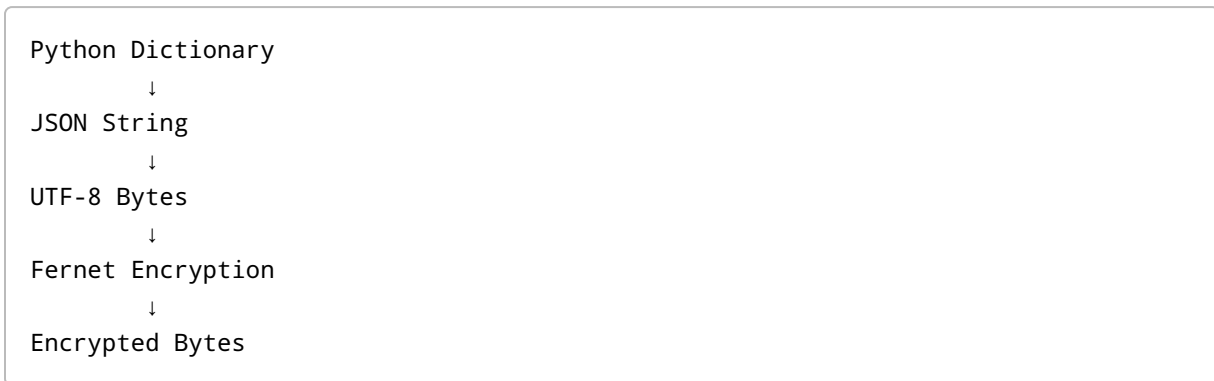
Aura checks whether the memory key exists.



This allows Aura to maintain access to previously encrypted memory between sessions.

10. Memory Encryption

Before memory is saved:



The encrypted data is then written to:

```
redx_memory.enc
```

11. Memory Decryption

When Aura starts:

```
graph TD; A[Check Memory File] --> B[Read Encrypted Data]; B --> C[Fernet Decryption]; C --> D[Decode UTF-8]; D --> E[Parse JSON]; E --> F[Restore Dictionary];
```

If decryption or JSON processing fails, Aura returns an empty dictionary.

12. Persistent Memory

Aura loads memory during application startup.

Conceptually:

```
graph TD; A[Start Aura] --> B[Load Encryption Key]; B --> C[Create Cipher]; C --> D[Load Memory File]; D --> E[Decrypt Memory]; E --> F[Restore Memory Dictionary];
```

This allows memory stored during previous sessions to remain available.

13. Command Processing

The command-processing system is handled by the main command function.

Commands are cleaned before processing.

```
Raw Input
  ↓
Remove Extra Outer Whitespace
  ↓
Create Lowercase Version
  ↓
Match Command Logic
  ↓
Execute Action
```

14. Supported Commands

Greeting

```
hello
```

Aura responds with a greeting.

The command detection checks whether the word appears within the submitted input.

Remember

Syntax:

```
remember <key> <value>
```

Example:

```
remember project Aura
```

Processing:

```
Split Command
  ↓
Extract Key
  ↓
Extract Value
  ↓
Update Memory Dictionary
  ↓
Save Encrypted Memory
```

Recall

Syntax:

```
recall <key>
```

Example:

```
recall project
```

Aura retrieves the value associated with the requested key.

If no value exists:

```
Not found.
```

is returned.

Activate Redx

Command:

```
activate redx
```

This launches the Redx authentication process.

15. Redx Mode

Redx Mode is an additional protected application state.

Activation requires successful decryption of the protected Redx key.

After activation:

```
redx_mode_active = True
```

Aura logs the activation and opens the Redx Command Center.

The current implementation also prevents redundant activation attempts when Redx Mode is already active.

16. Redx Command Center

The Redx Command Center is implemented as a separate Tkinter window.

The interface contains four buttons.

Tool 1: Digital Shield

Logs:

```
Digital Shield Activated.
```

Current status:

```
Prototype / Simulated Feature
```

Tool 2: Stealth Protocols

Logs:

```
Stealth Protocols Online.
```

Current status:

Prototype / Simulated Feature

Tool 3: Purge Memory

This performs a real memory operation.

Process:

```
Current Memory
  ↓
Clear Dictionary
  ↓
Encrypt Empty Dictionary
  ↓
Overwrite Memory File
```

The operation removes all currently stored Aura memory.

Tool 4: Threat Scan

Logs a threat-scanning status message.

Current status:

Prototype / Simulated Feature

The implementation does not perform actual operating system security analysis.

17. User Interface

Aura v2 uses a single main Tkinter application.

The interface includes:

- Application title
- Subtitle

- Command log
- Text input field
- Execute button
- Activate Redx Mode button

The main window is configured as:

520x460

The application identifies itself visually as:

AURA GEN 2

with the subtitle:

Encrypted Memory and Redx Mode

The UI configuration and startup command list are defined in the main application section.

18. Input Handling

Aura supports two methods of command execution.

Enter Key

The user can press:

Enter

to execute the current command.

Execute Button

The user can click:

Execute

to process the current command.

Both methods use the same command execution workflow.

19. Error Handling

Aura v2 includes basic handling for several error conditions.

Missing Protected Redx Key

If the protected key file cannot be found, Aura displays an error dialog.

Incorrect Redx Password

If decryption fails because the password-derived key cannot unlock the protected key, Aura displays an access-denied error.

Invalid Memory Data

If encrypted memory cannot be decrypted or parsed correctly, Aura returns an empty memory structure.

Missing Memory Key

If no memory key exists, Aura automatically generates a new one.

20. Security Analysis

Aura v2 represents a substantial improvement over Aura v1.

Improvements

Aura v2 introduces:

- Fernet symmetric encryption
 - Encrypted persistent memory
 - Separate encryption keys
 - Password-derived key processing
 - Protected Redx key storage
-

Current Limitations

The password derivation process uses direct SHA-256 hashing.

For a stronger production-grade authentication system, future versions should consider:

- Argon2
- scrypt
- PBKDF2
- Random salts
- Configurable iteration counts
- Authentication rate limiting

These mechanisms are better suited to resisting password-guessing attacks.

21. Failure and Recovery Considerations

Aura's encrypted memory depends on the associated memory key.

Conceptually:

```
redx_key.key
+
redx_memory.enc
=
Ability to Decrypt Memory
```

If the memory key is lost while encrypted memory remains, the stored memory may no longer be recoverable through the application.

This makes backups an important future consideration.

22. Comparison With Aura v1

Feature	Aura v1	Aura v2
GUI	Tkinter	Tkinter
Commands	Basic	Expanded
Memory	Temporary	Persistent

Feature	Aura v1	Aura v2
Encryption	No	Fernet
Memory Storage	RAM	Encrypted File
Authentication	Basic Password Check	Protected Key Decryption
Redx Mode	Command Mode	Dedicated Command Center
Project Structure	Primarily Single File	Multi-File
Cryptography	No	Yes

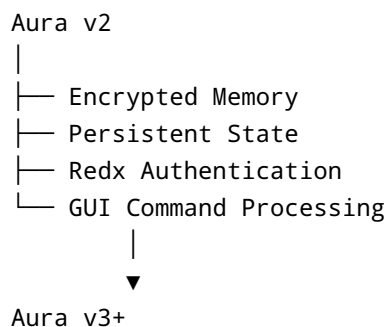
23. Current Limitations

Aura v2 does not currently include:

- AI language models
- Natural language understanding
- Voice input
- Text-to-speech
- Internet connectivity
- Real threat scanning
- Real digital shield functionality
- Plugin architecture
- Database storage
- User accounts
- Authentication rate limiting
- Advanced logging

24. Future Architecture

Potential Aura v3 development could introduce:



- |
- |— AI Model Integration
- |— Voice Interface
- |— Semantic Memory
- |— Modular Tools
- |— System Automation
- |— Secure Authentication
- |— Plugin System
- |— Advanced Context Management

25. Version Information

```
Project: Aura
Generation: 2
Version: v2.0
Primary Language: Python
GUI Framework: Tkinter
Encryption: Fernet
Hashing: SHA-256
Memory: Persistent
Memory Storage: Encrypted Local File
Authentication: Password-Based Protected Key
Redx Mode: Available
Internet Access: Not Implemented
AI Model: Not Implemented
```

26. Conclusion

Aura v2 is the point where Aura evolved beyond a simple command-response prototype.

The project introduced three major architectural concepts:

```
Persistent State
+
Encryption
+
Authentication
```

The encrypted memory system allows Aura to retain information between sessions, while Redx Mode introduces a separate authentication and protected-key mechanism.

Aura v2 is still a lightweight command assistant rather than a modern AI system, but it establishes the infrastructure required for more advanced future generations.

```
Aura v1
Basic Commands
+
Temporary Memory
    ↓
Aura v2
Persistent Memory
+
Encryption
+
Authentication
+
Redx Command Center
    ↓
Future Aura Generations
AI
+
Automation
+
Context
+
Advanced Tools
```

Aura v2 is the second major foundation of the Aura project.